A Dialectica Model of State

*Marcelo da S. Corrêa** *Edward H. Haeusler*
Dept. de Informática
Pontifícia Univ. Católica (PUC-Rio)
Rua Marquês de São Vicente, 225
Rio de Janeiro - RJ - Brazil - 22453-900
E-mail: {correa,hermann}@inf.puc-rio.br

Valeria C.V. de Paiva
Computer Laboratory
University of Cambridge
New Museums Site - Pembroke Street
Cambridge CB2 3QG - U.K.
E-mail: Valeria.Paiva@cl.cam.ac.uk

Abstract

Reddy introduced an extended intuitionistic linear calculus, called LLMS *(for Linear Logic Model of State), to model some features of state manipulation. His calculus includes the connective "before" and an associated modality †. De Paiva presents a (collection of) dialectica categorical models for Classical Linear Logic, the categories GC. These categories contain an extra tensor product functor $\odot$ and a comonad structure corresponding to a modality related to this tensor. It is surprising that these works arising from completely different motivations can be related in a meaningful way. In this paper, we adapt Reddy's system* LLMS *providing it with a commutative version of the connective "before", denoted by $\odot$, and an associated modality. We show that such a new conective can be interpreted as the interleaving operator of* CSP. *We also construct a dialectica category $\mathcal{G}$ on* **Sets** *and show that it is a sound model for the modified system* LLMS_c.

Keywords Intuitionistic Linear Logic, Proof Theory, Concurrency and Categorical Models.

1 Introduction

The concept of state-manipulation plays an important role in computation. There are many attempts at formalising this concept, especially in the framework of Girard's Linear Logic ([8, 10, 11, 17, 18, 15]). Some aspects of state-manipulation, like sequencing and regenerative values, are captured in an interesting way by Reddy's work[15]. Reddy presents an extended intuitionistic linear calculus for modelling state manipulation. His calculus LLMS (for Linear Logic Model of State) adds two extra connectives to those of Intuitionistic Linear Logic: a sequencing binary operator $\triangleright$, called "before", and a "regenerative" storage operator †, a modality associated with "before". These two constructors are used to express dynamic values and imperative programs within the framework of linear logic, while Girard's "of course" modality is used to express static values. The approach is shown to work by embbeding a higher-order Algol-like language in the system LLMS.

Reddy's intuitive idea is that the connective "before" captures the computational feature of sequential composition with possible one-way (left to right) communication. Thus this connective must be noncommutative. If we consider a commutative version of this connective, we could initially think of it as something like composition with possible two-way communication. However, we realized that it can represent concurrent execution without interaction or synchronization, i.e. *interleaving*. Interleaving is a basic concept in the implementation of concurrency (sometimes it is also useful in providing semantics cf. resumptions [13]).

In this paper we show how one could think of interleaving in the framework of LLMS. We introduce a new connective $\odot$ which is just the commutative version of Reddy's $\triangleright$ and we modify the system LLMS accordingly, obtaining a linear calculus, dubbed LLMS_c (section 2). We interpret the connective $\odot$ as the interleaving operator of CSP [9]. We also obtain distinct interpretations for the additive conjunction and the tensor

*Also: Dept. de Análise - Instituto de Matemática - Univ. Federal Fluminense

Proceedings of CATS'96 (Computing: The Australasian Theory Symposium), Melbourne, Australia, January 29–January 30 1996.

product using variations of the concurrency operator. In fact, we obtain a sound interpretation of the fragment of LLMS_c containing these connectives in the framework of CSP (section 3).

On the other hand, we had already a categorical model for LLMS_c, a *dialectica* style one. Dialectica models were introduced by de Paiva [3] as a model for the intuitionistic fragment of Linear Logic (the motivation for the model comes from Gödel's Dialectica Interpretation, hence the name). Later, de Paiva introduced the categories GC [4, 5], a different kind of dialectica model, which following a suggestion of Girard model the whole of Linear Logic. Actually the categories GC have some extra structure (an extra tensor product and an extra modality, associated with this tensor product) and hence can be seen as a model of LLMS_c. The extra (monoidal) comonad can be seen as an extra modality, which we think of as a commutative version of the "regenerative" storage operator. In the section 4, we describe a specific dialectica category $\mathcal{G}$ (built over Sets) and show that it is a sound model for LLMS_c. Detailed proofs of the results stated here can be found in [2].

2 The system LLMS_c

The presentation of the system of LLMS_c follows strictly the presentation of LLMS [15].

The left context of a sequent in LLMS_c has the following syntax :

$$\Gamma ::= \epsilon | A | \Gamma_0, \Gamma_1 | \Gamma_0; \Gamma_1$$

where ϵ (empty context) is interpreted as $\mathbf{1}$ and the contexts Γ_0, Γ_1 and $\Gamma_0; \Gamma_1$ are interpreted as $\Gamma_0 \otimes \Gamma_1$ and $\Gamma_0 \odot \Gamma_1$, respectively. A context Γ, which does not contain any occurrences of the connective ";", is called an *independent context*.

A context Γ is characterized as a pomset(partially ordered multiset) $(|\Gamma|, \leq_\Gamma)$, where $|\Gamma|$ is the set of formula occurrences in Γ, and $\leq_\Gamma$ is a partial order on $|\Gamma|$. They are defined inductively as follows:

$$\begin{array}{rcll} |\epsilon| & = & \{\} & \\ |A| & = & \{A\} & \leq_A = \{(A,A)\} \\ |\Gamma_1, \Gamma_2| & = & |\Gamma_1| + |\Gamma_2| & \leq_{\Gamma_1,\Gamma_2} = \leq_{\Gamma_1} \cup \leq_{\Gamma_2} \\ |\Gamma_1; \Gamma_2| & = & |\Gamma_1| + |\Gamma_2| & \end{array}$$

$$\leq_{\Gamma_1;\Gamma_2} = \leq_{\Gamma_1} \cup \leq_{\Gamma_2} \cup (|\Gamma_1| \times |\Gamma_2|) \cup (|\Gamma_2| \times |\Gamma_1|)$$

The rules of LLMS_c are just those of LLMS [15], except that we use the symbol $\odot$ instead of $\triangleright$. The fact that $\odot$ is commutative, whereas $\triangleright$ is not, is "hidden" in the definition of the contexts. The rules of LLMS_c are presented in the Figure 1.

Next, we describe some inferences which are derived from the structural rule and which show some interesting properties of the connectives $\otimes$ and $\odot$.

Structural Rule

$$\frac{\Gamma' \vdash A}{\Gamma \vdash A}\mathbf{Ser} \quad \text{if } |\Gamma| = |\Gamma'| \text{ and } \leq_\Gamma \subseteq \leq_{\Gamma'}$$

Identity Rules

$$\frac{}{A \vdash A}\mathbf{Id} \qquad \frac{\Gamma \vdash A \qquad \Delta[A] \vdash B}{\Delta[\Gamma] \vdash B}\mathbf{Cut}$$

Multiplicative Rules

$$\frac{\Gamma \vdash A \quad \Delta \vdash B}{\Gamma, \Delta \vdash A \otimes B} \otimes\mathcal{R} \qquad \frac{\Gamma[A, B] \vdash C}{\Gamma[A \otimes B] \vdash C} \otimes\mathcal{L}$$

$$\frac{\Gamma \vdash A \quad \Delta \vdash B}{\Gamma; \Delta \vdash A \odot B} \odot\mathcal{R} \qquad \frac{\Gamma[A; B] \vdash C}{\Gamma[A \odot B] \vdash C} \odot\mathcal{L}$$

$$\frac{}{\vdash \mathbf{1}}\mathbf{1}\mathcal{R} \qquad \frac{\Gamma[\epsilon] \vdash C}{\Gamma[\mathbf{1}] \vdash C}\mathbf{1}\mathcal{L}$$

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash A \multimap B} \multimap\mathcal{R} \qquad \frac{\Gamma \vdash A \quad \Delta[B] \vdash C}{\Delta[\Gamma, A \multimap B] \vdash C} \multimap\mathcal{L}$$

Additive Rules

$$\frac{}{\vdash \top}\top\mathcal{R} \qquad \frac{\Gamma \vdash A \quad \Gamma \vdash B}{\Gamma \vdash A \& B} \&\mathcal{R}$$

$$\frac{\Gamma[A] \vdash C}{\Gamma[A \& B] \vdash C}\mathbf{1}\&\mathcal{L} \qquad \frac{\Gamma[B] \vdash C}{\Gamma[A \& B] \vdash C}\mathbf{2}\&\mathcal{L}$$

Modalities

$$\frac{\Gamma[\epsilon] \vdash C}{\Gamma[\dagger A] \vdash C}\dagger\mathbf{Weak} \qquad \frac{\Gamma[A] \vdash C}{\Gamma[\dagger A] \vdash C}\dagger\mathbf{Der}$$

$$\frac{\Gamma[\dagger A; \dagger A] \vdash C}{\Gamma[\dagger A] \vdash C}\dagger\mathbf{Thread}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash \dagger A} \quad \text{(if } \Gamma \text{ is an independent context with only ! or † formulae)}$$

$$\frac{\Gamma[\dagger A] \vdash C}{\Gamma[!A] \vdash C}!\mathbf{Ser} \qquad \frac{\Gamma[!A, !A] \vdash C}{\Gamma[!A] \vdash C}!\mathbf{Contr}$$

$$\frac{\Gamma \vdash A}{\Gamma \vdash !A} \quad \text{(if } \Gamma \text{ is an independent context with only ! formulae)}$$

Figure 1: Proof system of LLMS_c

Proposition 2.1 *The following inferences are allowed by the Structural Rule (***Ser***).*

$$\textbf{(i)}\frac{\Delta[(A, B); C] \vdash D}{\Delta[A, (B; C)] \vdash D} \qquad \textbf{(ii)}\frac{\Delta[A; (B, C)] \vdash D}{\Delta[(A; B), C] \vdash D}$$

$$\textbf{(iii)}\frac{\Delta[A; B] \vdash C}{\Delta[A, B] \vdash C} \qquad \textbf{(iv)}\frac{\Delta[(A, C); (B, D)] \vdash E}{\Delta[(A; B), (C; D)] \vdash E}$$

Commutativity and associativity of the tensor products are also consequences of the structural rule. The *weak-distributivity* properties

$$A \otimes (B \odot C) \multimap (A \otimes B) \odot C$$
$$(A \odot B) \otimes C \multimap A \odot (B \otimes C)$$

follow from the inferences (**i**) and (**ii**), respectively. The inferences (**iii**) and (**iv**) allow us to prove that

$$A \otimes B \multimap A \odot B$$
$$(A \odot B) \otimes (C \odot D) \multimap (A \otimes C) \odot (B \otimes D)$$

We can obtain a Gentzen's style proof of the cut-elimination for LLMS_c.

Theorem 2.1 (Cut Elimination) *If a sequent is provable in* LLMS_c*, then it is provable in* LLMS_c *without an application of the cut rule.*

This proof is probably similar to Reddy's proof [14, 15], but we consider as a measure of the complexity, the rank as well as the degree of the last cut-formula, with the usual definitions. Special attention must be payed to the cases involving †, for instance, the case in which the †**Thread** is applied in the lower right sequent of a proof ending by a cut rule is proved using the inferences (**iii**) and (**iv**).

3 A fragment of LLMS_c in CSP

In this section, we present an interpretation of a fragment of LLMS_c including the operators $\odot$, the tensor product $\otimes$ and the additive conjunction $\&$ in the CSP (*Communicating Sequential Process* [9]) formalism, which gives an interesting description of computational processes.

We present intuitive descriptions of the concepts which we use here. We follow the definitions given by Hoare [9]:

- In CSP the word *process* stands for the behaviour pattern of objects. A description of such a behaviour pattern is given by means of a set of kinds of both events or actions adequately chosen. The set of events associated to an object is called its *alphabet*. The alphabet associated to an object, whose behaviour pattern is described by a process P, is denoted αP.
- $STOP_A$ is the process which never engages in any of the events of its alphabet A.
- $(x \rightarrow P)$ is the process which first engages in the event x and so behaves as described by P, provided that $x \in \alpha P$.
- A *trace* of the behaviour of a process is a finite sequence of symbols recording the events in which the process has engaged up to some moment in time.
- The most important operation on traces is *catenation*. The catenation of a trace s with a trace t, denoted by $s\hat{}t$, determines a trace by simply putting s and t together, in this order.
- A trace s is a *prefix* of a trace t ($s \leq t$), if s is a copy of an initial subsequence of t, i.e. there is an extension u of s such that the sequence constructed by the events of s followed by the events of u is the same as t. The empty trace $\langle\rangle$ is a prefix of any trace t.
- The sequential composition $(P;Q)$ of the processes P and Q is the process which first behaves like P; but when P terminates sucessfully, it continues by behaving like Q.
- The *concurrency* operator $||$ concerns interactions occurring between processes when they run concurrently. If it is applied to processes which have the same alphabet, it assumes that they interact with each other in a kind of lock-step synchronization. If it is applied to processes P and Q which have different alphabets, the events that are in both their alphabets require simultaneous participation of both P and Q; the events that are in αP but are not in αQ are no concern to Q, thus, P may engage in them independently of Q. Similarly, Q may engage alone in events which are in αQ but are not in αP.
- The *interleaving* operator $|||$ concerns concurrently combination of processes with the same alphabet, without directly interacting or synchronizing with each other. Each event of a trace of the interleaving of two processes is an event which one of them can engage in. However, if both processes can engage in a same event in a certain time, the choice of which of them engage in such an event is nondeterministic.

  Each trace s of the interleaving of two processes P and Q can be constructed from a trace s_1 of P and a trace s_2 of Q by mixing their events in a nondeterministic way. In this case, we say that s *interleaves* s_1 and s_2.
- We must emphasize that processes with differents alphabets can be converted in order to have the same alphabet without change the behaviour patterns that they describe. For instance, αP can be extended by including symbols which are only in αQ and we can obtain the process $P_{+(\alpha Q - \alpha P)} = (P \,||\, STOP_{(\alpha Q - \alpha P)})$ which has exactly the same traces of P, but $\alpha P_{+(\alpha Q - \alpha P)} = \alpha P \cup \alpha Q$.

  Let us denote $P_{+(\alpha Q - \alpha P)}$ as P_Q and $Q_{+(\alpha P - \alpha Q)}$ as Q_P

The denotation of the formulae in general will be given as processes in CSP.

The connective $\odot$ can be interpreted as interleaving. One motivation for this is the fact that we have lost the sequential aspect of the original connective "before" and gained some sort of non-ordered combination. We chose to give a general interpretation for the latter rather than a particular one, like composition with possible two-way communication. After all interleaving is a general and interesting combination of processes.

The linear tensor product is oftenly considered as representing absence of communication. The concept of communication of processes is represented in CSP by means of the *concurrency* operator ||, but two processes might communicate only if their alphabets have a common event. We could obtain a denotation for the tensor product by means of the concurrency operator applied to processes with disjoint alphabets. Thus, we must transform the original processes into processes with different alphabets. This can be done in CSP by labelling each process by a different name such that each event of a labelled process is also labelled by this name. To characterize the tensor product which drives two premisses into only one, we make the labelled processes run concurrently until one of them stops.

In CSP, a process P labelled by l is denoted by $l : P$ and an event of the alphabet of $l : P$ is denoted by a pair $l.a$, where a is an event of the alphabet of P. We define an operator $\bigotimes$ to give a denotation for the tensor product.

Definition 3.1 *Let P and Q be processes, l_1 and l_2 distinct labels and d a non-labelled event, such that $d \notin \alpha P \cup \alpha Q$. Then, the operator $\bigotimes$ is defined by*
$P \bigotimes Q = (l_1 : P; d \rightarrow STOP \,||\, l_2 : Q; d \rightarrow STOP)$

The additive conjunction & is usually considered as representing the intersection of elements. It is denoted here by the interaction of processes by means of the concurrency operator, since we must transform the original processes into processes with the same alphabet.

We compare the denotations of the formulae by means of a relation defined in terms of traces of processes. However, this relation is little more complex than would be expected, since the traces of the denotation of the tensor product are particularly differents from others: they have labelled events. We may have to deal with a trace s containing some events labelled by different labels $l_1, l_2, \cdots, l_n$. In spite of this, we can eliminate those labels obtaining a *label-free sequence* denoted $s \downarrow \{l_1, l_2, \cdots l_n\}$, as it is recursively defined below.

Definition 3.2 *Let s be a trace and L the set of labels which occur in s. Then we define the label-free sequence $s \downarrow L$ recursively by:*

(i) $\langle\rangle \downarrow L = \langle\rangle$

(ii) $(\langle l.a\rangle \hat{} s') \downarrow L = (\langle a\rangle \hat{} s') \downarrow L, \quad$ *if* $s = \langle l.a\rangle \hat{} s'$.

(iii) $(\langle a\rangle \hat{} s') \downarrow L = \langle a\rangle \hat{} (s' \downarrow L), \quad$ *if a is a non-labelled event and* $s = \langle a\rangle \hat{} s'$.

Now, we present formally the interpretation discussed above.

Definition 3.3 *Let* LLMS_c^* *be a fragment of* LLMS_c *which contains only the connectives $\otimes$, $\odot$ and &.*
Let $[\![\ ]\!]$ be a mapping, such that:

- $[\![A]\!]$ *is a process, if A is atomic;*
- $[\![A \otimes B]\!] = [\![A]\!] \bigotimes [\![B]\!]$
- $[\![A \& B]\!] = [\![A]\!]_{[\![B]\!]} \;||\; [\![B]\!]_{[\![A]\!]}$
- $[\![A \odot B]\!] = [\![A]\!]_{[\![B]\!]} \;|||\; [\![B]\!]_{[\![A]\!]}$
- $[\![\Gamma]\!] = [\![\gamma]\!]$,
  where the formula γ is obtained interpreting each occurrence of the operators "," and ";" in the context Γ as $\otimes$ and $\odot$, respectively;
- $\Gamma \vdash C$ *is interpreted as* $[\![\Gamma]\!] \preceq [\![C]\!]$
  where the relation $P \preceq Q$, reads P **is subsumed by** *Q, is defined by:*

  $P \preceq Q$ iff for each trace s_1 of P, there is a trace s_2 of Q, such that $s_1 \downarrow L_1 \;\leq\; s_2 \downarrow L_2$, where L_1 (L_2) is the set of labels which occur in s_1 (s_2, respectively).

We have, hence, the following.

Theorem 3.1 (Soundness) *The mapping $[\![\ ]\!]$ preserves the inference rules of* LLMS_c^*.

The proof consist of checking that the rules of LLMS_c^* are preserved by the interpretation presented here. For the sake of simplicity, when we deal with the relation $\preceq$ we treat directly with the label-free sequences instead of the original traces of the denotations considered.

We cannot proof a completeness result for this interpretation, because we can establish some relations like $[\![A]\!] \preceq [\![A \odot B]\!]$ which do not represent theorems of LLMS_c^*.

4 A Categorical Model for LLMS_c

To characterize LLMS_c categorically, we must consider an instantiation $\mathcal{G}$ of the (symmetric monoidal closed) categories GC, developed in [5]. As usual, the interpretation of the formulae is given by objects of the category $\mathcal{G}$ and proofs of LLMS_c are interpreted by morphisms of $\mathcal{G}$.

In the most general case a GC-like category is obtained from a symmetric monoidal closed category C with products, (and satisfying some other conditions [7]) but for purposes of this paper, our base category will be **Sets**. Readers interested in the (several already known) generalizations of this work should consult [7, 6].

Recall that the partial-order with two elements $\mathbf{2}$, where one should think of $\mathbf{0}$ as false and $\mathbf{1}$ as true, is a closed-poset. It has a tensor product ($\otimes$) and an internal-hom ($\multimap$) such that $a \otimes b \leq c$ iff $a \leq b \multimap c$. Actually $\otimes$ is the usual conjunction $\wedge$ and the internal-hom *linear* is the usual Heyting implication $\rightarrow$.

Definition 4.1 *The category $\mathcal{G}$ is described by:*
- *Objects are relations on* **Sets**, *that is maps $U \times X \xrightarrow{\alpha} \mathbf{2}$, written as $(U \overset{\alpha}{\longleftarrow\!\!\!+} X)$.*
- *A morphism from $(U \overset{\alpha}{\longleftarrow\!\!\!+} X)$ to $(V \overset{\beta}{\longleftarrow\!\!\!+} Y)$ consists a pair of maps, $f\colon U \to V$ and $F\colon Y \to X$, such that the following condition is satisfied*

$$\alpha(u, Fy) \leq \beta(fu, y)$$

where $\leq$ is the usual order in $\mathbf{2}$.

Thus there is a morphism (f, F) from α to β iff for all u in U and all y in Y, if $u\alpha F(y)$ then $f(u)\beta y$.

We usually depict a morphism (f, F) in $\mathcal{G}$ as follows:

$$\begin{array}{ccc} U & \overset{\alpha}{\longleftarrow\!\!\!+} & X \\ {\scriptstyle f}\downarrow & & \uparrow{\scriptstyle F} \\ V & \underset{\beta}{\longleftarrow\!\!\!+} & Y \end{array}$$

The next definition describes the (multiplicative and additive) structure of the category $\mathcal{G}$ we are interested in.

Definition 4.2 *Let $A = (U \overset{\alpha}{\longleftarrow\!\!\!+} X)$ and $B = (V \overset{\beta}{\longleftarrow\!\!\!+} Y)$ be objects of $\mathcal{G}$.*
- *The tensor bifunctor $\otimes\colon \mathcal{G} \times \mathcal{G} \to \mathcal{G}$ is given by*

$$A \otimes B = (U \times V \overset{\alpha\otimes\beta}{\longleftarrow\!\!\!+} X^V \times Y^U),$$

where $(u, v)\, \alpha \otimes \beta\, (f, g)$ iff $u\alpha f(v)$ and $v\beta g(u)$. Its unit is the object $I = (\mathbf{1} \overset{\iota}{\longleftarrow\!\!\!+} \mathbf{1})$, where ι is the identity relation on $\mathbf{1}$.
- *The tensor bifunctor $\odot\colon \mathcal{G} \times \mathcal{G} \to \mathcal{G}$ is given by*

$$A \odot B = (U \times V \overset{\alpha\odot\beta}{\longleftarrow\!\!\!+} X \times Y),$$

where $(u, v)\, \alpha \odot \beta\, (x, y)$ iff $u\alpha x$ and $v\beta y$. Its unit is the same object I above.
- *The internal-hom bifunctor $[-,-]_{\mathcal{G}}\colon \mathcal{G}^{op} \times \mathcal{G} \to \mathcal{G}$ is given by*

$$[A, B]_{\mathcal{G}} = (V^U \times X^Y \overset{\beta^\alpha}{\longleftarrow\!\!\!+} U \times Y),$$

where $(f, F)\beta^\alpha(u, y)$ iff $u\alpha x$ implies $v\beta y$. The relation β^α is given by the composition $V^U \times X^Y \times U \times Y \xrightarrow{\langle \pi_3, \pi_4, eval, eval\rangle} U \times Y \times V \times X \xrightarrow{`\alpha\times\beta'} \mathbf{2} \times \mathbf{2} \xrightarrow{\multimap} \mathbf{2}$.
- *The product bifunctor $\&\colon \mathcal{G} \times \mathcal{G} \to \mathcal{G}$ is given by*

$$A \& B = (U \times V \overset{\alpha\&\beta}{\longleftarrow\!\!\!+} X + Y),$$

where $(u, v)\, \alpha \& \beta \begin{pmatrix} x, 0 \\ y, 1 \end{pmatrix}$ iff either $u\alpha x$ or $v\beta y$. Its unit is the object $\top = (\mathbf{1} \overset{e}{\longleftarrow\!\!\!+} \mathbf{0})$.

We could also describe coproducts (or additive disjunctions), a par connective and linear negation, but these will not play an important role in the following discussion.

Lemma 4.1 *For any A and B objects of $\mathcal{G}$ there are natural morphisms:*

(i) $\tau_{(A,B)}: A \otimes B \to A \odot B$; [1]

(ii) $w_{(A,B,C)}: A \otimes (B \odot C) \to (A \otimes B) \odot C$ *(called weak-distributivity);*

(iii) $v_{(A,B,C,D)}: (A \odot B) \otimes (C \odot D) \to (A \otimes C) \odot (B \otimes D)$.

Recall that **Sets** does have free (commutative) monoid structures. There is an adjunction

$$F \dashv \mathrm{U}: \mathbf{Mon}_c \to \mathbf{Sets}$$

where $\mathbf{Mon}_c$ is the category of commutative monoids U is the forgetful functor and in

$$F(X) = (X^*, e_X, m_X)$$

X^* can be viewed as finite sequences (up to permutation) of elements of X, $e_X: \mathbf{1} \to X^*$ is the empty sequence and $m_X: X^* \times X^* \to X^*$ means concatenation of sequences. We need to consider another monad structure in **Sets.** Consider the monad $(T_U, \eta_{T_U}, \mu_{T_U})$, where for a given U in **Sets**, $T_U: \mathbf{Sets} \to \mathbf{Sets}$ is the endofunctor which takes $X \mapsto X^U$, $Y \mapsto Y^U$ and if $g: X \to Y$, $f \in X^U$ is taken to $f;g \in Y^U$. Its unit is given by the transpose of the projection, that is the constant map $(\eta_{T_U})_X: X \to X^U$ and its multiplication $(\mu_{T_U})_X: X^{U \times U} \to X^U$, is simply precomposition with the diagonal map on U.

We define two endofuctors on $\mathcal{G}$ to model the modalities of LLMS_c.

Definition 4.3 *Consider the following endofunctors on $\mathcal{G}$:*

1. *The functor $\dagger$ on $\mathcal{G}$ given on objects by $\dagger(U \overset{\alpha}{\leftarrow\!\!+} X) = (U \overset{\dagger\alpha}{\leftarrow\!\!+} X^*)$ where the relation $\dagger(\alpha)$ is defined by*

   $u(\dagger\alpha)(x_1, \cdots, x_k)$ *iff* $u \alpha x_1$ *and* $\cdots$ *and* $u \alpha x_k$

   and $\dagger$ acts on maps as $\dagger(f, F) = (f, F^*)$.

2. *The functor G on $\mathcal{G}$ given by*

   $$G(U \overset{\alpha}{\leftarrow\!\!+} X) = (U \overset{G\alpha}{\leftarrow\!\!+} X^U)$$

   where the relation $G\alpha$ is given by $u(G\alpha)f$ iff $u \alpha f(u)$. The functor G applied to a map $(f, F): A \to B$ is $(f, F(-)f): GA \to GB$.

[1] There is another natural morphism $\sigma_{A,B}: (A \odot B) \to (A \square B)$ where $\square$ is the functor correponding to the par connective, but it shall not concern us here.

Formally the relation $\dagger\alpha$ is given by the map $U \times X^* \xrightarrow{\dagger\alpha} \mathbf{2}$ which is the composition

$$U \times X^* \xrightarrow{r_{U,X}} (U \times X)^* \xrightarrow{\alpha^*} \mathbf{2}^* \longrightarrow \mathbf{2}$$

(the map $r_{U,X}: U \times X^* \to (U \times X)^*$ takes a tuple $\langle u, x_1, \ldots, x_k \rangle \mapsto \langle \langle u, x_1 \rangle, \ldots, \langle u, x_k \rangle \rangle$ and $\mathbf{2}^* \to \mathbf{2}$ using the tensor product in $\mathbf{2}$) and the relation $G\alpha$ is given by the composition

$$U \times X^U \xrightarrow{\langle \pi_1, eval \rangle} U \times X \xrightarrow{\alpha} \mathbf{2}$$

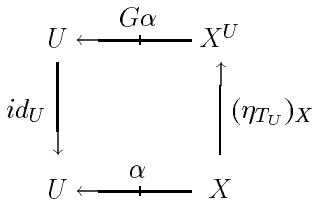
Lemma 4.2 *The functors $\dagger$ and G have natural (monoidal) comonad structures $(\dagger, \epsilon_\dagger, \delta_\dagger)$ and $(G, \epsilon_G, \delta_G)$, induced by the monad structure of $*$ and the monads T_U, respectively.*

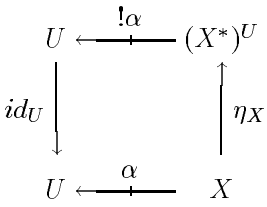
- *The counit $(\epsilon_\dagger)_A: \dagger A \to A$ is given by identity on U and the singleton map $X \overset{(\eta_*)_X}{\to} X^*$ and the comultiplication $(\delta_\dagger)_A: \dagger A \to \dagger^2 A$ is given by identity on U and forgetting brackets $X^{**} \overset{(\mu_*)_X}{\to} X^*$ in the second coordinate, as in the diagram*

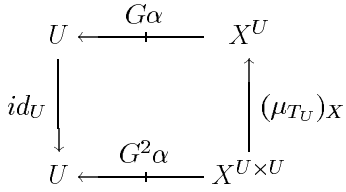
$$\begin{array}{ccc} U & \overset{\dagger\alpha}{\leftarrow\!\!+} & X^* \\ {\scriptstyle id_U}\downarrow & & \uparrow{\scriptstyle (\eta_*)_X} \\ U & \overset{\alpha}{\leftarrow\!\!+} & X \end{array}$$

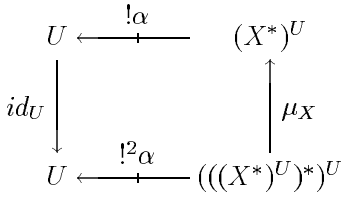
$$\begin{array}{ccc} U & \overset{\dagger\alpha}{\leftarrow\!\!+} & X^* \\ {\scriptstyle id_U}\downarrow & & \uparrow{\scriptstyle (\mu_*)_X} \\ U & \overset{\dagger^2\alpha}{\leftarrow\!\!+} & X^{**} \end{array}$$

- *The counit $(\epsilon_G)_A: GA \to A$ is given by identity on U and the natural constant map $(\eta_{T_U})_X: X \to X^U$ in the second coordinate; the comultiplication $(\delta_G)_A: GA \to G^2 A$ is given by identity on U and the restriction to the diagonal, $(\mu_{T_U})_X: X^{U \times U} \to X^U$, in the second coordinate:*



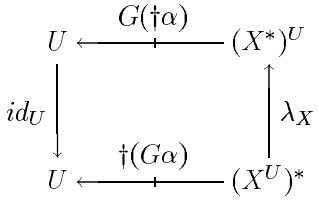






Now there is a distributive law relating our comonads † and G. Then, by abstract nonsense [1] we have that these comonads compose and that G lifts to the category of coalgebras for †.

Lemma 4.3 *There is a distributivity law of comonads $\Lambda_A: G(\dagger A) \to \dagger(GA)$ given by*



where each component $\lambda_X: (X^U)^* \to (X^*)^U$ of the natural transformation λ is given by the exponential transpose of the following composition

$$(X^U)^* \times U \xrightarrow{r_{X^U,U}} (X^U \times U)^* \xrightarrow{ev^*} X^*$$

Definition 4.4 *Consider the composite comonad $(G;\dagger)$ and call it !. Its functor part acts on objects as $!(U \xleftarrow{\alpha} X) = (U \xleftarrow{!\alpha} (X^*)^U)$ and on maps as $!(f,F) = (f, F^*(-)f)$. We have $(\epsilon_!)_A: !A \to A$ and $(\delta_!)_A: !A \to !!A$ as follows:*

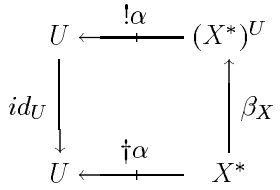
where η_X is given by $X \xrightarrow{(\eta_*)_X} X^* \xrightarrow{(\eta_{T_U})_X} (X^*)^U$ and μ_X is given by the exponential transpose of the long composition

$$(((X^*)^U)^*)^U \times U \xrightarrow{\langle ev,\pi\rangle} ((X^*)^U)^* \times U \xrightarrow{\lambda\times U} (X^{**})^U \times U \xrightarrow{ev} X^{**} \xrightarrow{(\mu_*)_X} X^*$$

As the name indicates it is this composite comonad that will play the role of the modality ! in Linear Logic. Next we emphasize some properties which are used in the proof of soundness of the model.

Proposition 4.1 *We have the following facts*

1. *There is a comonad morphism [2] $\kappa: ! \to \dagger$, such that $\kappa_A: !A \to \dagger A$ is given by*



where $\beta_X: X^* \to (X^*)^U$ is given by (η_{T_U}) applied to the object X^*.

[2] This morphism is similar to Reddy's Ser, but it is not a monomorphism in $\mathcal{G}$.

2. *For any $\mathcal{G}$-object A, $!A$ has a comonoid structure $(!A, d_A, e_A)$ with respect to $\otimes$ with $e_A: !A \to I$ and $d_A: !A \to !A \otimes !A$ as its counit and comultiplication, respectively. There is a functor from $\mathcal{G}$ to $\mathbf{Comon}_{\otimes}\mathcal{G}$ which takes A to $(!A, d_A, e_A)$.*

3. *For any $\mathcal{G}$-object A, $\dagger A$ has a comonoid structure $(\dagger A, d'_A, e'_A)$ with respect to $\odot$ with $e'_A: \dagger A \to I$ and $d'_A: \dagger A \to \dagger A \odot \dagger A$ as its counit and comultiplication, respectively. There is a functor $R_\dagger: \mathcal{G} \to \mathbf{Comon}_{\odot}\mathcal{G}$ which takes A to $(\dagger A, d'_A, e'_A)$. Moreover $R_\dagger$ is right-adjoint to the forgetful functor $\mathrm{U}: \mathbf{Comon}_{\odot}\mathcal{G} \to \mathcal{G}$.*

Theorem 4.1 (Soundness) *The symmetric monoidal closed category $\mathcal{G}$, with bifunctors internal-hom $[-,-]_{\mathcal{G}}$; tensor products $\otimes$ and $\odot$; the additive conjunction $\&$ and the comonads $!$ and $\dagger$, is a model for* LLMS_c*. Thus, each entailment $\Gamma \vdash_{LLMS_c} A$ corresponds to the existence of a morphism in $\mathcal{G}$, $(f, F): [\![\Gamma]\!] \to [\![A]\!]$.*

To prove the theorem, we have only to check that the rules of LLMS_c are preserved by the interpretation presented.

As we hinted before, the category has more structure than the necessary to model LLMS_c: for example we do have multiplicative and additive disjunctions in $\mathcal{G}$, as well as a linear negation. Also it is not clear at the moment which components of the structure of $\mathcal{G}$ are essential for modelling LLMS. In this connection it is interesting to investigate a fuller version of LLMS with the connectives above, which is work in progress.

5 Conclusions

The motivation for this work came from O'Hearn's and Reddy's use of Linear Logic to deal with the semantics of Algol-like programming languages. Earlier on we realised that we could use Reddy's ideas to give a syntactical characterization to some of the (somewhat mysterious) constructions of de Paiva's dialectica categorical models *GC* for Classical Linear Logic. In particular it was interesting to find an intuitive explanation for the comonad $\dagger$ in terms of the extra tensor product $\odot$.

Summing up the work so far, we adapted Reddy's system LLMS providing it with a commutative version of the connective "before" and an associated modality. Further we interpreted this new connective as the *interleaving*. A dialectica category $\mathcal{G}$ was constructed on **Sets** and shown to be a sound model of the new system LLMS_c.

But much work remains to be done. Firstly, going back to our original motivation, more work is needed to relate the system LLMS_c to O'Hearn's and Reddy's recent work[12, 16]. Secondly, from a logical pespective, we might want to use this system with two modalities, as a blueprint for work on relating modalities. Thirdly the interpretation of a fragment of LLMS_c in terms of CSP operators is useful, as it gives a very tangible explanation of our plethora of tensor products. But it would be more convincing if we could also explain the modalities and the linear implication in CSP terms. The domain-theoretical interpretation of the addressed fragment of LLMS_c can be built by using the standard event structure based semantics for CSP. Extending this to the whole of LLMS_c is also a topic for future work.

References

[1] J. Beck. Distributive laws. In *Seminar on Triples and Categorical Homoly Theory, LNM 80*, pages 119–140. Springer-Verlag, 1969.

[2] M. da S. Corrêa, E. H. Haeusler and V. C. V. de Paiva. A dialectica model of state. Technical Report PUC-Rio.MCC21/95, Dept. de Informática, Pontifícia Univ. Católica PUC-Rio, August 1995.

[3] V. C. V de Paiva. Dialectica categories. In *Categories in Computer Science and Logic*, Boulder, Colorado, June 1987. American Mathematical Society. (Contemporary Mathematics, Vol. 92).

[4] V. C. V de Paiva. *Dialectica categories*. Ph.D. thesis, Lucy Cavendish College, Cambridge, November 1988. Printed as Technical Report No. 213, University of Cambridge Computer Laboratory, January, 1991.

[5] V. C. V. de Paiva. A dialectica-like model of linear logic. In *Category Theory in Computer Science*, Volume 389 of *LNCS*, pages 341–356, September 1989.

[6] V. C. V. de Paiva. A category of multirelations. Technical Report 255, University of Cambridge Computer Laboratory, April 1991.

[7] V. C. V. de Paiva and J. M. E. Hyland. Generalising dialectica categories. (Revised version of a talk in the Montreal Category Theory Meeting 1991), 1995.

[8] J.-Y. Girard. Towards a geometry of interaction. In *Categories in Computer Science and Logic*, pages 69–108, Boulder, Colorado, June 1987. American Mathematical Society. (Contemporary Mathematics, Vol. 92).

[9] C. A. R. Hoare. *Communicating sequential processes*. Prentice-Hall International, London, 1985.

[10] Y. Lafont. The linear abstract machine. *Theoretical Computer Science*, Volume 59, pages 157–180, 1988.

[11] P. W. O'Hearn. Linear logic and interference control. In *Categories in Computer Science and Logic*, Volume 530 of *LNCS*, pages 74–93, Paris, France, September 1991. Springer-Verlag, Berlin.

[12] P. W. O'Hearn, A. J. Power, M. Takeyama and R. D. Tennent. Syntactic control of interference revisited. In *MFPS XI*, Volume 1 of *Eletronic Notes in Theoretical Computer Science*, New Orleans, March 1995.

[13] G. Plotkin. A power domain construction. *SIAM Journal of Computing*, Volume 5, pages 412–427, 1976.

[14] U. S. Reddy. A linear logic model of state (preliminar report). Eletronic manuscript (ftp), csc.uiuc.edu, directory /pub/reddy/papers, July 1992.

[15] U. S. Reddy. A linear logic model of state. Eletronic manuscript (ftp), csc.uiuc.edu, directory /pub/reddy/papers, October 1993.

[16] U. S. Reddy. Passivity and independence. In *9th Annual IEEE Symposium on Logic in Computer Science*, pages 342–352, Paris, France, 1994. IEEE Computer Society Press, Los Alamitos, California.

[17] P. Wadler. Linear types can change the world! In *Programming Concepts and Methods*, Amsterdam, 1990. North-Holland. (Proc. IFIP TC 2 Working Conf, Sea of Galilee, Israel).

[18] P. Wadler. Is there a use for linear logic? In *Proc. Symp. on Partial Evaluation and Semantics-Based Program Manipulation, ACM*, pages 255–273, 1991. (SIGPLAN Notices, Sep. 1991).